

Campus Student Guide Mobile Application

TEAM MEMBERS

DLAMINI LONDIWE 202304119 (BSc) - TEAM LEADER

MAGAGULA SETSEMBELE 202302550 (BSc) - DESIGNER

KUNENE WAZILE 202303918 (BSc) - ARCHITECTER

MASILELA MBONGISENI 202301022 (BSc) - PROGRAMMER

Table of Contents

Table of Contents.....	2
1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Overview	4
1.4 Reference Material	5
1.5 Definitions and Acronyms	5
2. System Overview	6
3. System Architecture	7
3.1 Architectural Design	7
3.1.1 Architectural Tiers	7
3.1.2 Architectural Diagram (Descriptive)	7
3.2 Decomposition Description	8
3.2.1 Mobile Client Modules	8
3.2.2 Backend API Modules	8
3.2.3 Sequence: Student Searches for a Building	8
3.3 Design Rationale	9
4. Data Design	10
4.1 Data Description	10
4.2 Data Dictionary	10
Table: Users	10
Table: Locations.....	10
Table: Events	11
Table: Notices	11
5. Component Design	13
5.1 AuthModule (Client)	13
Purpose	13
Interface.....	13
Processing Logic (Pseudocode).....	13
5.2 MapModule (Client)	13
Purpose	13
Interface.....	13
Processing Logic (Pseudocode).....	13
5.3 DirectoryModule (Client)	14
Purpose	14

Interface.....	14
Processing Logic (Pseudocode).....	14
5.4 AuthController (API Server)	14
Purpose	14
Interface.....	14
Processing Logic (Pseudocode).....	14
5.5 NoticesController (API Server).....	15
Purpose	15
Interface.....	15
Processing Logic (Pseudocode).....	15
6. Human Interface Design.....	16
6.1 Overview of User Interface	16
6.2 Screen Descriptions	16
6.2.1 Home Screen	16
6.2.2 Map Screen.....	16
6.2.3 Directory Screen	16
6.2.4 Events Screen.....	17
6.2.5 Login Screen.....	17
6.3 Screen Objects and Actions	17
7. Requirements Matrix	19
8. Appendices	21
Appendix A — Design Viewpoints (IEEE 1016-2009 Mapping).....	21
Appendix B — Change History	21
Appendix C — Glossary of Design Patterns Used	21
Appendix D — API Endpoint Summary.....	22

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the architecture, components, data structures, interfaces, and behavioural design of the Campus Student Guide Mobile Application. It translates the project's software requirements into a comprehensive design representation suitable for guiding implementation by the development team.

This document is prepared in conformance with IEEE Std 1016-2009, Software Design Descriptions. It is intended for the following audiences:

- Software Architects — for architectural decision validation
- Designers and Programmers — as the primary reference during implementation
- Testers — for deriving test cases and integration test plans
- Project Leaders and Quality Assurance Staff — for design review and verification

1.2 Scope

The Campus Student Guide Mobile Application is a cross-platform mobile app designed to assist university students in navigating and utilizing campus resources effectively. The application provides:

- Interactive campus maps with building locations, lecture halls, laboratories, and amenities
- An academic event and timetable management system
- A searchable campus directory covering staff, departments, and services
- Real-time notices and announcements
- Integration with existing university portals and databases

The system targets Android and iOS platforms and is designed to serve students, academic staff, and administrative personnel at the university. The scope of this SDD covers the mobile client application, the backend REST API service, and the data management layer.

1.3 Overview

This document is organized in accordance with the SDD template derived from IEEE Std 1016-2009 and contains the following major sections:

- Section 1 — Introduction: Purpose, scope, references, and definitions
- Section 2 — System Overview: General system description and context
- Section 3 — System Architecture: Architectural design and decomposition
- Section 4 — Data Design: Data structures and data dictionary
- Section 5 — Component Design: Detailed module and component descriptions
- Section 6 — Human Interface Design: UI overview, screen layouts, and interactions
- Section 7 — Requirements Matrix: Traceability from requirements to design components
- Section 8 — Appendices: Supporting diagrams and supplementary material

1.4 Reference Material

Ref #	Document	Source
[1]	IEEE Std 1016-2009: Software Design Descriptions	IEEE Computer Society
[2]	IEEE Std 830-1998: Recommended Practice for SRS	IEEE
[3]	Campus Student Guide — Software Requirements Specification	Project Team
[4]	IEEE Std 12207-2008: Software Life Cycle Processes	IEEE
[5]	React Native Documentation v0.73	Meta Open Source
[6]	Node.js / Express.js API Documentation	OpenJS Foundation

1.5 Definitions and Acronyms

Term / Acronym	Definition
SDD	Software Design Document — the current document
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer — architectural style for networked APIs
UI / UX	User Interface / User Experience
CRUD	Create, Read, Update, Delete — basic data operations
JWT	JSON Web Token — used for stateless authentication
GPS	Global Positioning System — used for location services
MVC	Model-View-Controller — architectural pattern
PoI	Point of Interest — a marked campus location
DB	Database
ORM	Object Relational Mapper — database abstraction layer

2. System Overview

The Campus Student Guide Mobile Application is a student-centric information and navigation platform that consolidates campus resources into a single mobile interface. The system addresses the persistent challenge faced by new and returning students in locating campus facilities, keeping track of academic events, and accessing up-to-date institutional information.

The application operates as a thin client that communicates with a cloud-hosted backend over a RESTful API. Core capabilities include:

- **Interactive Map Navigation:** Students can explore a zoomable campus map, search for specific buildings or points of interest, and receive step-by-step walking directions between locations.
- **Campus Directory:** A searchable database of lecturers, departments, administrative offices, and support services, complete with contact details and office locations.
- **Events and Timetables:** Display of academic events, class schedules, and institutional announcements synchronized from the university's administrative systems.
- **Notices and Alerts:** Push-notification delivery of urgent campus announcements, maintenance alerts, and schedule changes.
- **User Profile and Personalization:** Students can log in via university credentials, save favourite locations, and configure notification preferences.

The system comprises three primary tiers: the Mobile Client (React Native), the Backend API Server (Node.js/Express), and the Data Layer (PostgreSQL relational database). The mobile client communicates exclusively with the backend API; no direct database access is permitted from the client. Authentication is managed via JWT tokens issued by the API on successful login against the university identity provider.

The system is designed to degrade gracefully — core map data and previously cached directory information remain accessible offline. Synchronization occurs automatically when network connectivity is restored.

3. System Architecture

3.1 Architectural Design

The Campus Student Guide Application follows a three-tier client-server architecture with a clear separation between presentation, business logic, and data persistence layers. This architecture was selected for its scalability, maintainability, and alignment with the team's technical expertise.

3.1.1 Architectural Tiers

- Tier 1 — Presentation (Mobile Client): React Native application targeting Android and iOS. Responsible for all UI rendering, local caching, GPS-based location services, and push notification reception. Communicates with Tier 2 exclusively via HTTPS REST calls.
- Tier 2 — Business Logic (API Server): Node.js application using the Express framework. Implements all business rules, authentication/authorisation logic, and data transformation. Exposes a versioned RESTful API. Connects to Tier 3 via an ORM (Sequelize).
- Tier 3 — Data Layer (Database): PostgreSQL relational database that stores all persistent system data. Accessed only by the API server. Includes structured data for users, locations, events, notices, and directory entries.

3.1.2 Architectural Diagram (Descriptive)

The following describes the key subsystems and their interconnections:

Subsystem	Responsibility	Technology
Mobile Client	UI, GPS, local cache, push notifications	React Native
API Gateway	Request routing, rate limiting, HTTPS termination	Express Router
Auth Service	Login, JWT issuance, session management	JWT / bcrypt
Map Service	Serve map tiles, Poi data, route calculation	Custom module
Directory Service	Staff and department search and retrieval	Express + ORM
Events Service	Academic timetable and event management	Express + ORM
Notices Service	Campus announcements and push notifications	Express + FCM
Database	Persistent relational data storage	PostgreSQL

3.2 Decomposition Description

The system is decomposed into the following primary modules. An object-oriented decomposition is used, reflecting the React Native component model on the client and the service-oriented structure on the backend.

3.2.1 Mobile Client Modules

- **NavigationModule:** Manages in-app screen routing using React Navigation. Controls transitions between Home, Map, Directory, Events, Notices, and Profile screens.
- **MapModule:** Renders the interactive campus map. Integrates react-native-maps for tile display. Handles Poi markers, user location overlay, and route polyline rendering.
- **DirectoryModule:** Provides search and display of campus staff and services. Implements debounced search input and paginated result rendering.
- **EventsModule:** Fetches and displays academic timetable and event data. Supports calendar view and list view. Implements local reminder scheduling.
- **NoticesModule:** Displays campus announcements. Manages Firebase Cloud Messaging (FCM) token registration and push notification handling.
- **AuthModule:** Handles login form, credential submission, JWT storage (SecureStore), and session expiry management.
- **CacheModule:** Implements offline-first caching using AsyncStorage. Manages cache invalidation and background synchronization.

3.2.2 Backend API Modules

- **AuthController:** Validates credentials against the university identity provider, issues JWT tokens, and handles token refresh.
- **MapController:** Serves Poi records, building metadata, and walking-route data from the database.
- **DirectoryController:** Implements full-text search on staff and department records. Returns paginated results.
- **EventsController:** CRUD operations for academic events. Filters by date, department, and event type.
- **NoticesController:** CRUD for campus notices. Triggers FCM push notifications on new notice creation.
- **UserController:** Manages user profile data, favourite locations, and notification preferences.
- **MiddlewareLayer:** JWT authentication middleware applied to all protected routes. Request logging and error-handling middleware.

3.2.3 Sequence: Student Searches for a Building

1. Student enters building name in MapModule search bar.
2. MapModule calls GET /api/v1/locations?query={name} on the API server.

3. API Gateway authenticates JWT, forwards to MapController.
4. MapController queries the Locations table via ORM, returns matching Poi records.
5. MapModule renders marker on map and displays building info card.

3.3 Design Rationale

The three-tier client-server architecture was selected over alternatives for the following reasons:

- Separation of concerns: Business rules are centralised on the API server, making them testable and maintainable independently of the client.
- Cross-platform reach: React Native enables a single codebase targeting both Android and iOS, reducing development effort by an estimated 40% compared to native implementations.
- Offline resilience: The caching strategy at the client tier allows core functionality to persist during periods of poor campus connectivity.
- Security: No direct database access from the client eliminates SQL injection attack vectors at the mobile tier. JWT authentication avoids session state on the server.
- Scalability: The stateless API server can be horizontally scaled behind a load balancer without session affinity requirements.

A monolithic backend was chosen over microservices for the initial version given the team size and 18-week project timeline. The service module boundaries within the monolith are designed to be extractable to microservices in a future phase without refactoring client-facing API contracts.

4. Data Design

4.1 Data Description

All persistent application data is stored in a PostgreSQL relational database. The database is accessed exclusively through the Sequelize ORM on the API server. The primary data domains are:

- Users — Student and staff accounts, authentication credentials, preferences
- Locations — Campus buildings, rooms, outdoor spaces, and Points of Interest
- Directory Entries — Staff profiles, department information, contact details
- Events — Academic events, class timetable entries, deadlines
- Notices — Campus announcements with status and target audience
- Favourites — Junction table linking users to their saved locations

Map tile data is served as static assets from a CDN and is not stored in the relational database. Route calculations are performed on the API server using a pre-computed campus graph stored as a JSON adjacency list file, loaded into memory on server startup.

4.2 Data Dictionary

Table: Users

Field	Type	Constraints	Description
user_id	UUID	PK, NOT NULL	Unique user identifier
student_number	VARCHAR(20)	UNIQUE, NOT NULL	University student/staff number
full_name	VARCHAR(120)	NOT NULL	User's full name
email	VARCHAR(255)	UNIQUE, NOT NULL	Institutional email address
password_hash	VARCHAR(255)	NOT NULL	bcrypt-hashed password
role	ENUM	NOT NULL	'student', 'staff', 'admin'
fcm_token	VARCHAR(255)	NULLABLE	Firestore push notification token
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Account creation timestamp

Table: Locations

Field	Type	Constraints	Description
location_id	UUID	PK, NOT NULL	Unique location identifier
name	VARCHAR(200)	NOT NULL	Human-readable location name
category	ENUM	NOT NULL	'building', 'room', 'outdoor', 'service'
latitude	DECIMAL(10,7)	NOT NULL	GPS latitude coordinate
longitude	DECIMAL(10,7)	NOT NULL	GPS longitude coordinate
description	TEXT	NULLABLE	Description of the location
image_url	VARCHAR(500)	NULLABLE	URL to location thumbnail image
operating_hours	JSONB	NULLABLE	JSON object of daily opening hours

Table: Events

Field	Type	Constraints	Description
event_id	UUID	PK, NOT NULL	Unique event identifier
title	VARCHAR(200)	NOT NULL	Name of the event
description	TEXT	NULLABLE	Full event description
event_type	ENUM	NOT NULL	'lecture', 'exam', 'seminar', 'social'
start_time	TIMESTAMP	NOT NULL	Event start date and time
end_time	TIMESTAMP	NOT NULL	Event end date and time
location_id	UUID	FK -> Locations	Venue of the event
created_by	UUID	FK -> Users	Staff member who created the event

Table: Notices

Field	Type	Constraints	Description
notice_id	UUID	PK, NOT NULL	Unique notice identifier
title	VARCHAR(200)	NOT NULL	Notice headline
body	TEXT	NOT NULL	Full notice content

Field	Type	Constraints	Description
priority	ENUM	NOT NULL	'low', 'medium', 'high', 'urgent'
target_audience	ENUM	NOT NULL	'all', 'students', 'staff'
published_at	TIMESTAMP	NULLABLE	Scheduled or actual publish time
expires_at	TIMESTAMP	NULLABLE	Time after which notice is archived
created_by	UUID	FK -> Users	Admin or staff author

5. Component Design

This section provides detailed design specifications for each major component identified in Section 3.2. Each component is described using its purpose, inputs, outputs, processing logic, and error handling.

5.1 AuthModule (Client)

Purpose

Manages user authentication on the mobile client. Handles credential submission, token storage, session expiry, and logout.

Interface

- Input: student_number (string), password (string) from login form
- Output: JWT access token stored in SecureStore; navigation redirect to HomeScreen on success
- Output on failure: Error message displayed on login form

Processing Logic (Pseudocode)

```
FUNCTION login(studentNumber, password):  
    VALIDATE that studentNumber and password are non-empty  
    POST /api/v1/auth/login with { studentNumber, password }  
    IF response.status == 200:  
        STORE response.token in SecureStore  
        SET global auth context with user profile data  
        NAVIGATE to HomeScreen  
    ELSE IF response.status == 401:  
        DISPLAY 'Invalid credentials. Please try again.'  
    ELSE:  
        DISPLAY 'Service unavailable. Please retry later.'
```

5.2 MapModule (Client)

Purpose

Renders the interactive campus map with PoI markers and supports location search and walking-route display.

Interface

- Input: GPS coordinates from device location services; search query string from user
- Output: Rendered map with building markers, info cards, and route polylines

Processing Logic (Pseudocode)

ON component mount:

REQUEST device location permission
FETCH /api/v1/locations to populate markers
IF fetch fails: LOAD from CacheModule

FUNCTION onSearchSubmit(query):

FETCH /api/v1/locations?query=query
ANIMATE map to first result coordinates
RENDER result markers and info card

5.3 DirectoryModule (Client)

Purpose

Provides searchable access to staff profiles and department information.

Interface

- Input: Search query string (debounced, 300 ms)
- Output: Paginated list of matching staff/department cards

Processing Logic (Pseudocode)

FUNCTION onQueryChange(query):

IF query.length >= 2:
DEBOUNCE 300ms then FETCH /api/v1/directory?q=query&page=currentPage
RENDER results list; UPDATE pagination controls

5.4 AuthController (API Server)

Purpose

Validates login credentials, issues JWT tokens, and manages token refresh.

Interface

- POST /api/v1/auth/login — Input: { studentNumber, password }; Output: { token, user }
- POST /api/v1/auth/refresh — Input: Bearer refresh token; Output: { token }
- POST /api/v1/auth/logout — Input: Bearer token; Output: 204 No Content

Processing Logic (Pseudocode)

FUNCTION login(req, res):

FIND user WHERE student_number = req.body.studentNumber
IF user NOT FOUND: RETURN 401 Unauthorized
COMPARE req.body.password with user.password_hash using bcrypt
IF mismatch: RETURN 401 Unauthorized

SIGN JWT with { user_id, role } and 24h expiry
RETURN 200 { token, user: { user_id, full_name, role } }

5.5 NoticesController (API Server)

Purpose

Manages campus notice CRUD operations and triggers push notifications via Firebase Cloud Messaging (FCM) when high-priority notices are published.

Interface

- GET /api/v1/notices — Returns paginated active notices
- POST /api/v1/notices — Admin only; creates notice and optionally triggers push
- DELETE /api/v1/notices/:id — Admin only; archives a notice

Processing Logic (Pseudocode)

FUNCTION createNotice(req, res):

VALIDATE req.body fields

INSERT notice record into DB

IF notice.priority IN ['high', 'urgent']:

FETCH all user FCM tokens from Users table

SEND FCM multicast message with notice title and body

RETURN 201 { notice_id }

6. Human Interface Design

6.1 Overview of User Interface

The application follows Material Design 3 guidelines adapted for a university context. The colour palette uses the institution's primary blue (#1F3864) and white, ensuring brand consistency and accessibility compliance with WCAG 2.1 AA contrast ratios.

Navigation is implemented using a bottom tab bar with five primary destinations:

- Home — Dashboard showing recent notices, upcoming events, and quick-access shortcuts
- Map — Full-screen interactive campus map
- Directory — Staff and services search
- Events — Academic calendar and timetable
- Profile — User settings, favourites, and logout

Common UI patterns across all screens include pull-to-refresh for data synchronization, skeleton loaders during network requests, and a consistent error/empty state with actionable retry prompts.

6.2 Screen Descriptions

6.2.1 Home Screen

The Home Screen displays a personalized greeting using the authenticated user's name. Below the greeting, a horizontal scroll card section shows active urgent notices. A vertical list follows with upcoming events for the next 7 days. A quick-action grid at the bottom provides one-tap access to Map, Directory, and Events. A floating badge appears when new notices arrive while the screen is active.

6.2.2 Map Screen

The Map Screen renders the full campus using react-native-maps tiles. A search bar is pinned at the top. The user's current GPS location is shown as a pulsing blue dot. Campus buildings appear as colour-coded markers: academic buildings (blue), administrative buildings (grey), student services (green), and amenities (orange). Tapping a marker displays a bottom sheet info card with building name, operating hours, image, and a 'Get Directions' button. Directions display a polyline route from the user's location to the destination.

6.2.3 Directory Screen

The Directory Screen presents a search bar at the top and two tabs: People and Departments. The People tab returns staff members matching the search query, displaying name, title, department, and email. The Departments tab returns department cards with contact number, location, and HOD name. Tapping a result opens a detail screen. Tapping a location name in the detail screen deep-links to that building's map marker.

6.2.4 Events Screen

The Events Screen presents a monthly calendar view at the top with dots indicating days with events. Below, a list shows events for the selected date. Each event card displays title, time, location, and event type badge. Tapping a card opens an event detail screen with full description, venue map pin, and an 'Add to Reminders' option that schedules a local device notification.

6.2.5 Login Screen

The Login Screen is the entry point for unauthenticated users. It presents the university logo, two input fields (Student/Staff Number and Password), a Login button, and a 'Forgot Password?' link that directs users to the university's password reset portal in an in-app browser. Biometric login (fingerprint/Face ID) is offered on subsequent sessions if the device supports it.

6.3 Screen Objects and Actions

Screen	UI Object	Action / Behaviour
Login	Login Button	Submits credentials to AuthController; shows loading spinner
Login	Password Field	Toggleable visibility icon; masked by default
Home	Notice Card	Tap opens notice detail modal overlay
Home	Event List Item	Tap navigates to Event Detail screen
Map	Search Bar	Text input triggers debounced API query; clears with X icon
Map	PoI Marker	Tap displays bottom sheet info card with Get Directions option
Map	Get Directions Button	Renders polyline route from user GPS to selected location
Directory	Search Bar	300 ms debounce; fetches matching People or Departments
Directory	Staff Card	Tap opens Staff Detail screen; email is tappable mailto link
Events	Calendar Day	Tap selects day; event list below updates to show day's events
Events	Add to Reminders	Schedules local device notification 30 min before event start
Profile	Favourites List	Tapping a saved location deep-links to its map marker

Screen	UI Object	Action / Behaviour
Profile	Logout Button	Clears JWT from SecureStore; navigates to Login Screen

7. Requirements Matrix

The following traceability matrix maps each functional requirement from the Software Requirements Specification (SRS) to the design components that satisfy it. This ensures complete coverage of all stated requirements in the design.

Req ID	Requirement Description	Design Component(s)	Section
FR-01	Users shall be able to log in using university credentials	AuthModule, AuthController	5.1, 5.4
FR-02	System shall issue JWT tokens valid for 24 hours	AuthController, JWT middleware	5.4
FR-03	Users shall view an interactive campus map	MapModule, MapController	5.2, 3.2
FR-04	Users shall search for campus buildings by name	MapModule, MapController, Locations DB	5.2, 4.2
FR-05	System shall provide walking directions between locations	MapModule (route polyline), Campus graph	5.2, 3.1
FR-06	Users shall search the staff directory	DirectoryModule, DirectoryController	5.3, 3.2
FR-07	Users shall view academic events and timetables	EventsModule, EventsController, Events DB	3.2, 4.2
FR-08	Users shall set reminders for events	EventsModule (local notifications)	5.2, 6.3
FR-09	Admins shall publish campus notices	NoticesController, Notices DB	5.5, 4.2
FR-10	High-priority notices shall trigger push notifications	NoticesController, FCM integration	5.5
FR-11	Users shall save favourite locations	UserController, Favourites DB table	3.2, 4.1
FR-12	App shall function with limited network connectivity	CacheModule, AsyncStorage	3.2.1
FR-13	System shall support Android and iOS platforms	React Native cross-platform architecture	3.1
NFR-01	API response time shall be under 500 ms for 95% of requests	API Gateway, DB indexing strategy	3.1
NFR-02	Passwords shall be hashed with bcrypt (cost factor ≥ 12)	AuthController	5.4

Req ID	Requirement Description	Design Component(s)	Section
NFR-03	UI shall meet WCAG 2.1 AA accessibility contrast ratios	All UI components (Section 6)	6.1

8. Appendices

Appendix A — Design Viewpoints (IEEE 1016-2009 Mapping)

The following table maps this SDD's content to the formal design viewpoints defined in IEEE Std 1016-2009, Clause 5, confirming standard compliance.

IEEE 1016 Viewpoint	Design Concern Addressed	SDD Section
Context (5.2)	System boundary, actors, use cases	Sections 2, 3.1
Composition (5.3)	Subsystem decomposition and module structure	Section 3.2
Logical (5.4)	Class structure and relationships	Sections 3.2, 4.2 (entity model)
Dependency (5.5)	Module dependencies and integration order	Section 3.2
Information (5.6)	Persistent data structures and data dictionary	Section 4
Interface (5.8)	API contracts and UI interaction definitions	Sections 5, 6.3
Interaction (5.10)	Sequence of operations and messaging	Section 3.2.3
Algorithm (5.12)	Procedural logic for key components	Section 5

Appendix B — Change History

Version	Date	Author	Description of Change
0.1	April 2026	Project Team	Initial draft — system overview and architecture sections
0.2	May 2026	Project Team	Added data design, component design, and UI sections
1.0	May 2026	Project Leader	First complete version submitted for review

Appendix C — Glossary of Design Patterns Used

- Repository Pattern — Used in all controllers to abstract database access behind service interfaces, improving testability.

- Observer Pattern — Used in the NoticesModule to subscribe to FCM events and update the UI without polling.
- Strategy Pattern — Used in the CacheModule to switch between network-first and cache-first fetch strategies based on connectivity status.
- Singleton — The auth context is implemented as a React Context singleton accessible throughout the component tree.

Appendix D — API Endpoint Summary

Method	Endpoint	Auth Required	Description
POST	/api/v1/auth/login	No	Authenticate and receive JWT
GET	/api/v1/locations	Yes	List all PolS (with optional ?query=)
GET	/api/v1/locations/:id	Yes	Get single location details
GET	/api/v1/directory	Yes	Search staff/departments (?q=, ?type=)
GET	/api/v1/events	Yes	List events (?date=, ?type=)
GET	/api/v1/notices	Yes	List active campus notices
POST	/api/v1/notices	Admin	Create notice + optional push trigger
GET	/api/v1/users/me	Yes	Get authenticated user profile
PUT	/api/v1/users/me/favourites	Yes	Add/remove favourite location